1 Breve introducción a R
2 Variables
3 Operaciones básicas
4 Vectores
5 Matrices
6 data.frames
7 Ejercicios
8 Extra: Listas

Clase prácticas. Día 1: Introducción a R

Juan A. Ferrer-Bonsoms

1 Breve introducción a R

1.1 Descargar R y Rstudio

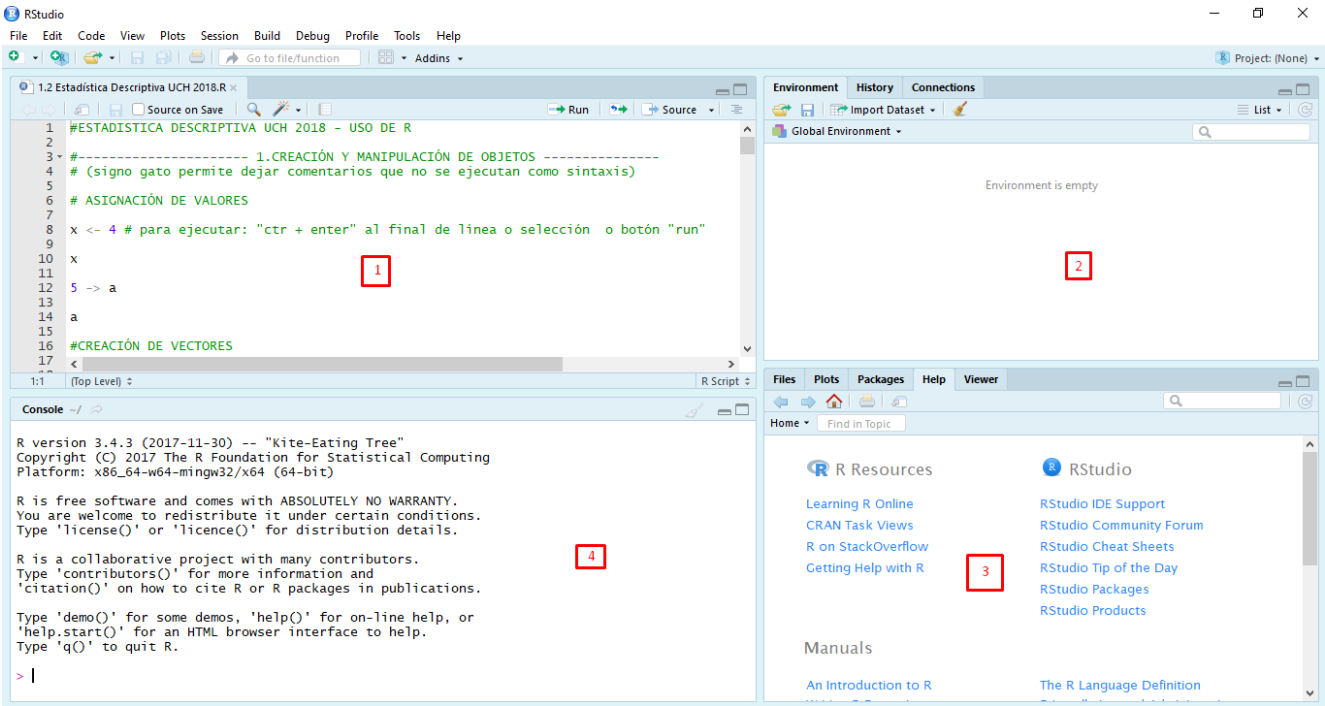
Para descargar R hay que ir al siguiente enlace: https://cran.r-project.org/.

Para descargar Rstudio hay que ir al siguiente enlace: https://www.rstudio.com/products/rstudio/download/

Es importante descargar **primero R y luego Rstudio**.

1.2 Rstudio

En la siguiente imagen tienen la interfaz de RStudio:



Interfaz gráfica (taken from https://bookdown.org/gboccardo/manual-ED-UCH/uso-basico-de-rstudio.html)

- (1): es el editor de scripts: se trata del lugar donde editamos nuestro código para posteriormente ejecutarla. Al escribir allí no sucederá nada, a no ser que se apriete algún botón para ejecutar los comandos o la tecla **ctrl+enter**. Al pulsar ctrl+enter se ejecutaran las lineas seleccionadas o la línea donde se encuentre el cursor.
- (2): es el "entorno de trabajo" del programa: en este lugar se muestra el conjunto de datos y los "objetos" (resultados, variables, gráficos, etc.) que se almacenan al ejecutar diferentes análisis.
- (3): tiene varias sub pestañas: (i) la pestaña **files** permite ver los archivos que 'cuelgan' de la carpeta de nuestro 'working directory'; (ii) la pestaña **plots** permite visualizar los gráficos que se generen; (iii) la pestaña

packages permite ver los paquetes descargados y guardados en el disco duro así como gestionar su instalación o actualización; (iv) la ventana **help** permite acceder al help de las funciones si ejecutamos el comando `?help(function)` y también acceder al CRAN - Comprehensive R Archive Network (siempre que se cuente con conexión a Internet), página oficial del software que ofrece diferentes recursos para el programa: manuales para el usuario, cursos online, información general, descarga de paquetes, información de los paquetes instalados, etc.

- (4): es la consola. Corresponde a lo que sería el software R en su versión básica. Allí el software ejecuta las operaciones realizadas desde el editor de sripts.

¿Cómo trabajar con R?

Para trabajar de forma ordenada en R, es bueno tener un directorio de trabajo ('working directory') donde ir guardando los códigos que vamos a ir ejecutando. También es bueno tener en este mismo directorio un sitio donde cargar los archivos, imágenes que se necesiten para ejecutar nuestro código o para guardar los archivos que se obtengan como resultado de nuestro código.

ver el working directory

Con el comando `getwd()` R devuelve el directorio donde 'estamos trabajando':

```
getwd()
```

R devolverá la ruta (path) correspondiente a nuestro 'working directory'. Rstudio toma en cada ordenador una ruta concreta como 'working directory' por defecto. A esta ruta, R la resume en: "~/".

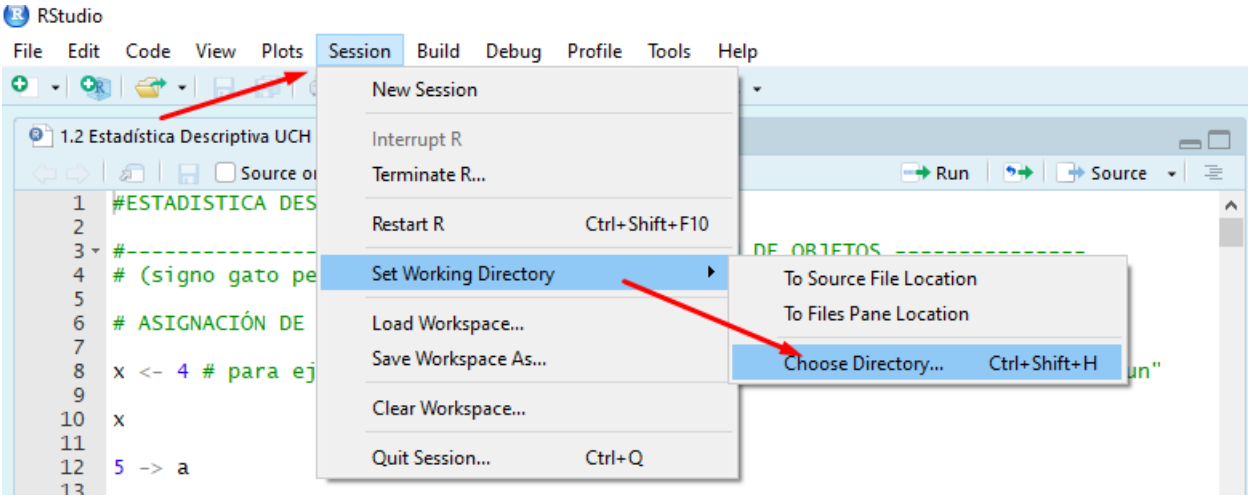
Cambiar el directorio de trabajo (working directory)

Existen **dos opciones** para cambiar el directorio de trabajo:

- utilizando el comando `setwd("ruta hasta carpeta donde quiero trabajar")`

```
setwd("~/practicas/practica_1_ejercicios_r/")
#como se ha explicado arriba, el directorio de trabajo por defecto en
#R puede ser por ejemplo: "C:/users/nombre/documents".
#Este path R lo abrevia con "~/".
#El código sería equivalente a:
# setwd("C:/users/nombre/documents/practicas/practica_1_ejercicios_r/")
```

- utilizando la pestaña **Session -> Set Working Directory -> Choose Directory -> examinar** (y se selecciona la carpeta a utilizar)



cambiar working directory (taken from https://bookdown.org/gboccardo/manual-ED-UCH/uso-basico-de-rstudio.html)

2 Variables

En R existen muchos tipos de variables, pero principalmente hay 2 tipos:

- Números.
- Texto.

En los siguientes apartados vamos a ver diferentes variables que podemos crear en R como vectores y matrices que contienen números o texto.

Finalmente vamos a ver un tipo de variable llamado `data.frame` que puede contener tanto números como texto.

```
# Números
num1 <- 20
num2 <- 3.5


# Texto
text1 <- 'Hello world!'
print(text1) #print es una función para mostrar en pantalla.
```

```
## [1] "Hello world!"
```

3 Operaciones básicas

El siguiente código muestra como sumar, restar, multiplicar y dividir:

```
# Operaciones
num1 + num2 #Suma
```

```
## [1] 23.5
```

```
num1 - num2 #Resta
```

```
## [1] 16.5
```

```
num1 * num2 #Producto
```

```
## [1] 70
```

```
num1/num2   #División
```

```
## [1] 5.714286
```

Otras operaciones, como calcular el producto matricial, irán apareciendo conforme las vayamos necesitando.

4 Vectores

4.1 crear un vector

Existen diferentes maneras de crear un vector:

- Función `c`:

```
#vector de numeros
MyVector<-c(3,6,9,12)
MyVector
```

```
## [1]  3  6  9 12
```

```
#vector de letras
MyVector_character<-c("a","b","c")
MyVector_character
```

```
## [1] "a" "b" "c"
```

No podemos construir un vector que tenga números y letras. Si lo intentamos, R convertirá los números en texto:

```
#vector de letras
MyVector_character<-c("a","b","c",34)
MyVector_character
```

```
## [1] "a"  "b"  "c"  "34"
```

- operador `:`

```
MyVector<-1:10
MyVector
```

```
##  [1]  1  2  3  4  5  6  7  8  9 10
```

- También se puede utilizar la función `seq`. La función `seq` que tiene tres parámetros: `from` (inicio), `to` (final) y `by` (incremento)

```
MyVector<-seq(from=5,to=25,by=5)
MyVector
```

```
## [1]  5 10 15 20 25
```

- La función `rep`

```
MyVector<-rep(10,5)
MyVector
```

```
## [1] 10 10 10 10 10
```

- también se puede crear un vector de otros dos vectores:

```
vector_1 <- c(1,2,3)
vector_2 <- 4:10
vector_3 <- c(vector_1,vector_2)
```

Más ejemplos:

```
# Combinaciones de todo lo anterior
sec10 <- c(seq(1,5), rep(20,5), 10)

# Comandos sirven para crear vectores
Week <- c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday",
          "Sunday")
Year <- c(rep(Week, 52))
```

Otra cosa importante respecto a trabajar con vectores es poner nombres a los valores del vector utilizando la función `names()`

```
Hours<-c(5,5,7,3,1,6,6)
names(Hours)<-c("Monday","Tuesday","Wednesday","Thursday","Friday","Saturday","Sunday")
Hours
```

```
##    Monday   Tuesday Wednesday  Thursday    Friday  Saturday    Sunday
##         5         5         7         3         1         6         6
```

accedecer a un valor concreto de un vector

con el operador `[]` podemos acceder a un elemento de un vector:

```
mi_vector <- c(1,2,3,4,2:10)
mi_vector
```

```
##  [1]  1  2  3  4  2  3  4  5  6  7  8  9 10
```

```
mi_vector[4] # elemento 4.
```

```
## [1] 4
```

```
mi_vector[1:4] # 4 primeros elementos.
```

```
## [1] 1 2 3 4
```

```
mi_vector[c(1,5)] #elemento 1 y 5.
```

```
## [1] 1 2
```

```
mi_vector[-4] #todos los elementos menos el cuarto
```

```
##  [1]  1  2  3  2  3  4  5  6  7  8  9 10
```

```
mi_vector[-c(1,10)] #todos los elementos menos el 1 y el 10
```

```
##  [1]  2  3  4  2  3  4  5  6  8  9 10
```

También se puede acceder según el nombre que tenga cada elemento del vector:

```
Hours[1]
```

```
## Monday
##      5
```

```
Hours["Monday"]
```

```
## Monday
##      5
```

5 Matrices

5.1 crear una matriz

La función principal para crear matrices en R es: `matrix(data,nrow,ncol,byrow)`

Los argumentos de entrada son

- **data** : Valores que van a formar la matriz.
- **nrow** : número de filas.
- **ncol** : número de columnas.
- **byrow** : si la matriz de llena por filas (TRUE) o por columnas (FALSE).

```
Mat<-matrix(1:9,nrow=3,ncol=3,byrow=T)
Mat
```

```
##      [,1] [,2] [,3]
## [1,]    1    2    3
## [2,]    4    5    6
## [3,]    7    8    9
```

```
Mat2<-matrix(1:9,nrow=3,ncol=3,byrow=F)
Mat2
```

```
##      [,1] [,2] [,3]
## [1,]    1    4    7
## [2,]    2    5    8
## [3,]    3    6    9
```

También se puede utilizar los comandos `cbind()` y `rbind()` para crear matrices a partir de otras matrices o de

vectores:

```
vector_1 <- c(1,2,3,4,5)
vector_2 <- 6:10
Mat3 <- cbind(vector_1,vector_2)
Mat3
```

```
##      vector_1 vector_2
## [1,]        1        6
## [2,]        2        7
## [3,]        3        8
## [4,]        4        9
## [5,]        5       10
```

```
vector_1 <- c(1,2,3,4,5)
vector_2 <- 6:10
Mat4 <- rbind(vector_1,vector_2)
Mat4
```

```
##          [,1] [,2] [,3] [,4] [,5]
## vector_1    1    2    3    4    5
## vector_2    6    7    8    9   10
```

```
Mat5 <- cbind(Mat3,Mat3)
Mat5
```

```
##      vector_1 vector_2 vector_1 vector_2
## [1,]        1        6        1        6
## [2,]        2        7        2        7
## [3,]        3        8        3        8
## [4,]        4        9        4        9
## [5,]        5       10        5       10
```

```
Mat6 <- rbind(Mat3,Mat3)
Mat6
```

```
##       vector_1 vector_2
##  [1,]        1        6
##  [2,]        2        7
##  [3,]        3        8
##  [4,]        4        9
##  [5,]        5       10
##  [6,]        1        6
##  [7,]        2        7
##  [8,]        3        8
##  [9,]        4        9
## [10,]        5       10
```

como acceder a elementos de una matriz

Al igual que con vectores, hay que utilizar los corchetes para acceder a elementos concretos de una matriz: `[filas,columnas]`:

```
Mat6[1:5,1:2] #filas 1 a 5 y  columnas 1 y 2
```

```
##      vector_1 vector_2
## [1,]        1        6
## [2,]        2        7
## [3,]        3        8
## [4,]        4        9
## [5,]        5       10
```

```
Mat6[1:5,] #filas 1 a 5 y todas las columnas
```

```
##      vector_1 vector_2
## [1,]        1        6
## [2,]        2        7
## [3,]        3        8
## [4,]        4        9
## [5,]        5       10
```

```
Mat6[,1:2] #todlas las filas y columnas 1 y 2
```

```
##       vector_1 vector_2
##  [1,]        1        6
##  [2,]        2        7
##  [3,]        3        8
##  [4,]        4        9
##  [5,]        5       10
##  [6,]        1        6
##  [7,]        2        7
##  [8,]        3        8
##  [9,]        4        9
## [10,]        5       10
```

```
Mat6[1,1:2] #fila 1 y columnas 1 y 2
```

```
## vector_1 vector_2
##        1        6
```

```
Mat6[1:5,1] #filas 1 a 5 y columna 1
```

```
## [1] 1 2 3 4 5
```

Si solo seleccionamos una columna o una fila, R nos devolverá un vector. Para que R mantenga las dimensiones de matriz, tenemos que añadir un parámetro `drop=FALSE`:

```
Mat6[1:5,1]
```

```
## [1] 1 2 3 4 5
```

```
Mat6[1:5,1,drop=FALSE]
```

```
##      vector_1
## [1,]        1
## [2,]        2
## [3,]        3
## [4,]        4
## [5,]        5
```

5.2 operaciones con matrices

Existen muchas funciones que se pueden hacer con matrices. Las principales que vamos a utilizar son:

- `%*%` Producto matricial.
- `*` Producto elemento a elemento entre dos matrices con la misma dimensión.
- `rownames` para mostrar o determinar el nombre de las filas.
- `colnames` para mostrar o determinar el nombre de las columnas.

A modo de ejemplo:

```
A = matrix(1:9,nrow=3)
B = matrix(2:10,nrow=3)

prod_a_b = A %*% B
prod_a_b
```

```
##      [,1] [,2] [,3]
## [1,]   42   78  114
## [2,]   51   96  141
## [3,]   60  114  168
```

Si quremos poner nombres a las filas y columnas:

```
rownames(prod_a_b) = c("r1","r2","r3")
colnames(prod_a_b) = c("c1","c2","c3")
```

Y si queremos mostrar el nombre de las filas y columnas:

```
rownames(prod_a_b)
```

```
## [1] "r1" "r2" "r3"
```

```
colnames(prod_a_b)
```

```
## [1] "c1" "c2" "c3"
```

6 data.frames

Los data.frames son estructuras de dos dimensione que contienen información que puede ser de diferente tipo. **EN LOS VECTORES Y LAS MATRICES TODOS LOS DATOS DEBEN SER DEL MISMO TIPO**.

En un data.frame **cada columna debe contener información del mismo tipo**, pero podemos tener columnas con texto y otras columnas con números.

6.1 Crear un data.frame

```
MyDataFrame <- data.frame(
  nombre = c("Josu","Ignacio","Ane Miren","Angel"),
  edad = c(18,19,21,23),
  otras_variables = c(25,35,12,36)
)

MyDataFrame
```

```
##      nombre edad otras_variables
## 1      Josu   18              25
## 2   Ignacio   19              35
## 3 Ane Miren   21              12
## 4     Angel   23              36
```

La forma de acceder a los elementos de un data.frame es igual que con una matriz. Pero existe la opción de utilizar el operador \(\$\) para acceder a los elementos de cada columna de la siguiente manera:

```
MyDataFrame[,1] #todas las filas y primera columna
```

```
## [1] "Josu"      "Ignacio"   "Ane Miren" "Angel"
```

```
MyDataFrame$nombre #primera columna entera
```

```
## [1] "Josu"      "Ignacio"   "Ane Miren" "Angel"
```

6.2 Modificar el data.frame:

- Añadir una nueva columna:

El siguiente código muestra dos formas diferentes de añadir una columna nueva a un data.frame existente:

```
#primera forma
MyDataFrame <- data.frame(MyDataFrame,nuevacolumna=c(1,2,3,4))
MyDataFrame
```

```
##      nombre edad otras_variables nuevacolumna
## 1      Josu   18              25            1
## 2   Ignacio   19              35            2
## 3 Ane Miren   21              12            3
## 4     Angel   23              36            4
```

```
#utilzando el operador $:
MyDataFrame$nueva_columna_2 <- c(T,T,F,F)
MyDataFrame
```

```
##      nombre edad otras_variables nuevacolumna nueva_columna_2
## 1      Josu   18              25            1             TRUE
## 2   Ignacio   19              35            2             TRUE
## 3 Ane Miren   21              12            3            FALSE
## 4     Angel   23              36            4            FALSE
```

- Eliminar una columna:

El siguiente código elimina la columna 4 y la columna 5.

```
MyDataFrame <- MyDataFrame[,-c(4,5)]
MyDataFrame
```

```
##      nombre edad otras_variables
## 1      Josu   18              25
## 2   Ignacio   19              35
## 3 Ane Miren   21              12
## 4     Angel   23              36
```

- Eliminar una fila:

El siguiente código elimina la fila 2.

```
MyDataFrame <- MyDataFrame[-2,]
MyDataFrame
```

```
##      nombre edad otras_variables
## 1      Josu   18              25
## 3 Ane Miren   21              12
## 4     Angel   23              36
```

- Añadir una fila

El siguiente código añade una fila nueva utilizando la función `rbind`:

```
MyDataFrame <- rbind(MyDataFrame,c("Ignacio",19,35))
MyDataFrame
```

```
##       nombre edad otras_variables
## 1       Josu   18              25
## 3  Ane Miren   21              12
## 4      Angel   23              36
## 41   Ignacio   19              35
```

7 Ejercicios

7.1 Ejercicios con vectores

1. Crear un vector que contenga valores del 1 al 100 con un incremento de 1. Este vector guardarlos en una variable de nombre A.
2. Crear un vector que contenga valores del 1 al 100 con un incremento de 2.Este vector guardarlos en una variable de nombre B.
3. del vector anterior seleccionar los 10 primeros valores.
4. del vector A sacar todos los valores menos los 10 primeros.
5. aplicar las siguientes funciones al vector A: `sqrt()`, `length()`, `order()`, `sort()` ¿Qué hace cada función? para ver que hace cada función puedes ver la ayuda con el comando `?function`
6. Sumar 5 a todos los valores del vector A.
7. Multiplicar A por si misma.

Extra. Los siguientes ejercicios se resuelven con funciones que veremos el próximo día:

8. calcular la media de B (utilizando la función `mean()`)
9. calcular la mediana de B (utilizando la función `median()`)

10. calcular la varianza de B (utilizando la función `var()`)
11. calcular la desviación típica de B (utilizando la función `sd()`)

7.2 Ejercicios con matrices

1. crear una matriz con valores del 1:12 que tenga 3 filas y 4 columnas.
2. utilizar la función `dim` para comprobar las dimensiones de la matriz.
3. utilizando las funciones `rownames` y `colnames` añadir nombre a las filas y las columnas.
4. utilizando las funciones `rownames` y `colnames` ver el nombre de las filas y de las columnas.
5. Calcular la raiz cuadrada de todos los valores de la matriz A.
6. Dividir todos los valores de la matriz A ente 12.
7. Multiplicar la matriz A por si misma elemento a elemento.
8. Con la función `t()` calcular la matriz traspuesta de A y guardarla en una variable B.
9. multiplicar matricialmente con el operador `%*%` la matriz A y B.
10. seleccionar todos los los elementos de la fila 1 de la matriz A (manteniendo el formato de matriz)
11. seleccionar todos los los elementos de la columna 3 de la matriz A (manteniendo el formato de matriz)
12. seleccionar todas las filas y todas las columnas menos la 2 de la matriz A.
13. seleccionar todas las columnas y todas las filas menos la 3 de la matriz A.
14. crear un vector con los valores 7,2 y 1 y guardarlo en una variable de nombre v1. Crear un vector con los valores 0 3 y -1 y guardarlo en una variable de nombre v2. Crear un vector con los valores -3, 4 y -2 y guardarlo en una variable de nombre v3. Juntar los tres vectores anteriores con la función `rbind` para crear una matriz de nombre D.
15. Calcular la matriz inversa de D con la función `solve`.

Extra. Los siguientes ejercicios se resuelven con funciones que veremos el próximo día:

16. Calcular la media de cada fila utilizando la función `rowMeans`
17. Calcular la media de cada columna utilizando la función `colMeans`
18. calcular la suma de todas las filas de la matriz A (utilizando la función `rowSums`)
19. calcular la suma de todas las columnas de la matriz A (utilizando la función `colSums`)

7.3 Ejercicios con data.frame

1. Crear un data.frame que contenga 4 columnas y 20 filas. La primera columna se debe llamar 'letras' y debe ser una columna de strings (utilizar el comando `letters[1:20]` para generar letras). La segunda columna debe llamarse 'Boolean' y debe contener datos boleanos (puede utilizar el siguiente comando: `rep(c(T,F),10)`). Y la tercera columna ('Numeros') debe contener números aleatorios: `rnorm(20,mean=0,sd=1)`
2. con el comando `$` acceder a todas las columnas. Utilizar la función `class()` para comprobar el tipo de variable de cada columna.
3. Acceder a la fila cuyo valor en la primera columna sea "d". Puede utilizar la función `which` para ver el índice de la fila deseada: `which(midataframe$letras == "d")`
4. Accede a las filas cuyo valor en la columna 'Boolean' sea `T`.
5. Eliminar la segunda columna.
6. Eliminar todas las filas en las cuales su valor en la columna 'Numeros' sea menor que 0.
7. Comprobar las dimensiones del data.frame resultante.

8 Extra: Listas

8.1 Crear una lista

Con la función `list` podemos crear una lista que concatena diferentes variables que pueden ser de diferente tipo:

```
A <- c(2, 3, 5)
B <- c("aa", "bb", "cc", "dd", "ee")
D <- c(TRUE, FALSE, TRUE, FALSE, FALSE)
E <- matrix(1:20,ncol=5)

MyList <- list(A,B,D,E)
MyList
```

```
## [[1]]
## [1] 2 3 5
##
## [[2]]
## [1] "aa" "bb" "cc" "dd" "ee"
##
## [[3]]
## [1]  TRUE FALSE  TRUE FALSE FALSE
##
## [[4]]
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    1    5    9   13   17
## [2,]    2    6   10   14   18
## [3,]    3    7   11   15   19
## [4,]    4    8   12   16   20
```

Al igual que con los vectores y las matrices, podemos añadir nombre a los elementos de una lista:

```
names(MyList)<-c("Numerical","Character","Boolean","Matrix")
MyList
```

```
## $Numerical
## [1] 2 3 5
##
## $Character
## [1] "aa" "bb" "cc" "dd" "ee"
##
## $Boolean
## [1]  TRUE FALSE  TRUE FALSE FALSE
##
## $Matrix
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    1    5    9   13   17
## [2,]    2    6   10   14   18
## [3,]    3    7   11   15   19
## [4,]    4    8   12   16   20
```

Acceder a los elementos de una lista

Para acceder a los elementos de una lista tenemos que utilizar el doble corchete: `[[indice]]`. Es decir, para acceder al primer elemento tendremos que poner el siguiente código:

```
MyList[[1]]
```

```
## [1] 2 3 5
```

8.2 Ejercicios con listas

1. Crear una matriz que contenga valores de 1 al 9 y que tenga 3 filas y 3 columnas y guardara en una variable de nombre A. Crear un vector que contenga 3 números y guardarlo en una variable de nombre v1. Crear un vector con 4 númeors y guardarlo en una variable de nombre v2. Crear una lista llamada MiLista que contenga la matriz A, el vector v1 y el vector v2.
2. Comprobar el nombre de los elementos de la lista.
3. Acceder al primer elemento de la lista (la matriz A) y añadirle una cuarta fila que contenga la información del vector v1 (utilizar la función `rbind`). Como resultado se obtendrá una matriz de 4 filas y 3 columnas. Guardar esta matriz en la lista creada en el lugar de la matriz A.
4. Al primer elemento de a lista (la matriz A, que ahora debe tener 4 filas y 3 columnas), añadirle una cuarta columna que contenga la información del vector v2 (utilizar la función `cbind`).
5. utilizar la función `length` para comprobar el tamaño de la lista.
6. utilizar la función `length` para comprobar el tamaño de los dos vectores que contiene la lista.
7. utilizar la función `dim` para comprobar el tamaño de la matriz que contiene la lista.
8. con el comando `$` acceder a los elementos de la lista. Ejemplo: `MiLista$A` accederá al elemento de la lista con nobmre `A`.